

Projekt zaliczeniowy — Algorytmy i Struktury Danych II

Królestwo Krasnoludków

Rok akademicki	2025/2026
Autor	Danila Kozik
Prowadzący	Kwiatkowska Anna
Repozytorium Git	github.com/kozik77/AISD-2 (dostęp dla prowadzącego nadany)
Termin oddania	30 maja 2026

1. Skład zespołu i role poszczególnych osób

Projekt realizowany jest samodzielnie, jako jednoosobowy „zespół”. W odróżnieniu od pozostałych grup zdecydowałem się pracować w pojedynkę — daje mi to pełną kontrolę nad architekturą i tempem prac, choć oczywiście kosztem braku osoby do code review oraz konieczności samodzielnego znalezienia każdego błędu.

W ramach jednoosobowej pracy odgrywam wszystkie role:

- **Projektant rozwiązania** — analiza treści zadania, odsianie warstwy fabularnej, dobór algorytmów i struktur danych.
- **Programista** — implementacja wszystkich czterech modułów w Pythonie.
- **Tester** — pisanie testów jednostkowych w pytest, generator danych losowych, testy integracyjne.
- **Dokumentalista** — niniejsza dokumentacja oraz komentarze w kodzie i README.
- **„Maintainer” repozytorium** — prowadzenie repozytorium Git, rozsądny podział na branche pomimo bycia jedyną osobą pchającą commity.

W przypadku braku zespołu szczególnie istotne stało się dla mnie pisanie dobrych testów i prowadzenie zwięzłych notatek w trakcie pracy — bo nie ma drugiej osoby, która zapyta „a dlaczego ten warunek tu jest?”.

2. Tytuł projektu

„Królestwo Krasnoludków — zintegrowany system optymalizacji pracy, patrolu granic i archiwizacji wiedzy z wykorzystaniem MCMF, otoczki wypukłej, RMQ i kodowania Huffmana”

Nazwa robocza pakietu w repozytorium: snowwhite_suite.

3. Specyfikacja problemu

Po odrzuceniu warstwy fabularnej zadanie sprowadza się do czterech podproblemów algorytmicznych, połączonych przepływem danych — wynik modułu 1 jest wejściem dla modułu 2, a wynik modułu 2 jest wykorzystywany przez moduł 3.

Moduł 1 — Przydział krasnoludków do wyrobisk

Wejście: zbiór krasnoludków $K = \{k_1, \dots, k_n\}$, zbiór wyrobisk $W = \{w_1, \dots, w_m\}$ z pojemnościami c_i , zbiór typów minerałów, dla każdego krasnoludka lista minerałów które potrafi wydobywać, dla każdego wyrobiska — typ minerału, oraz macierz odległości $D[i][j]$ pomiędzy domem k_i a wyrobiskiem w_j .

Wyjście: funkcja przydziału $f: K \rightarrow W \cup \{\perp\}$ taka, że liczba przydzielonych krasnoludków jest **maksymalna**, a wśród wszystkich takich przydziałów suma $\sum_i D[i][f(k_i)]$ jest **minimalna**, przy zachowaniu pojemności wyrobisk.

Moduł 2 — Trasa patrolu księcia

Wejście: zbiór punktów $P \subset \mathbb{R}^2$ — współrzędne wszystkich **użytkowanych** wyrobisk (tzn. tych, do których przydzielono co najmniej jednego krasnoludka).

Wyjście: uporządkowana lista wierzchołków otoczki wypukłej $\text{conv}(P)$ oraz długość jej obwodu.

Moduł 3 — Wyszukiwanie najgłośniejszego dekametrowca

Wejście: statyczna tablica $G[0..L-1]$ zawierająca głośności krasnoludków rozstawionych co stałą liczbę metrów wzdłuż trasy patrolu, oraz seria zapytań (l, r) .

Wyjście: dla każdego zapytania — indeks $i \in [l, r]$, dla którego $G[i]$ jest maksymalne.

Moduł 4 — Archiwizacja wiedzy

Wejście: dokument tekstowy zawierający opisy procedur i zgromadzoną wiedzę królestwa.

Wyjście: skompresowana reprezentacja dokumentu (kod Huffmana) oraz indeks umożliwiający wyszukiwanie słów kluczowych w czasie proporcjonalnym do długości wzorca (Trie zbudowane na zbiorze haseł).

4. Podział problemu na podproblemy i osoby odpowiedzialne za ich realizację

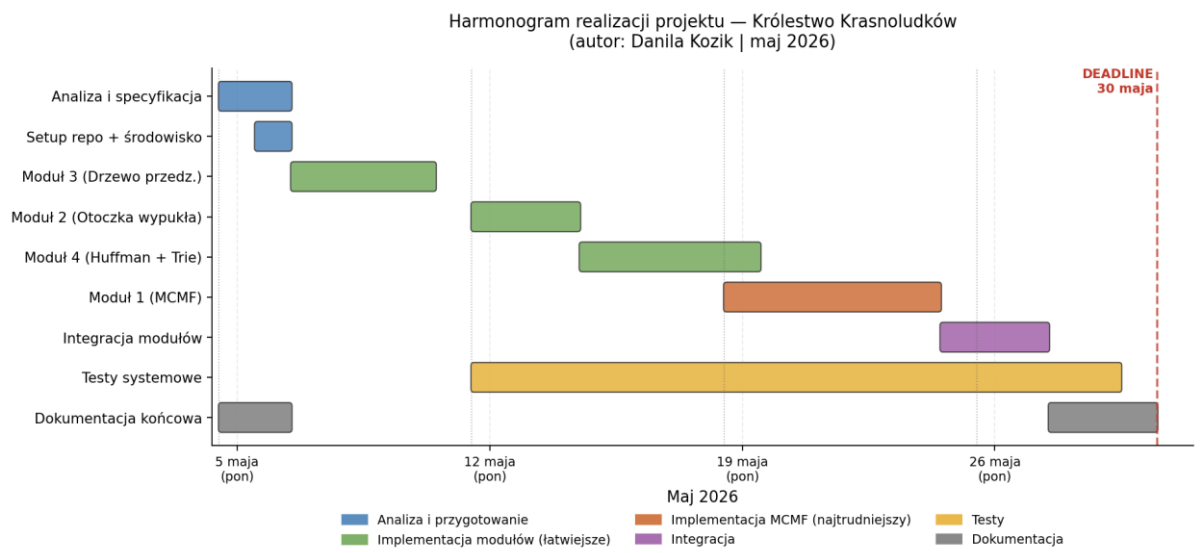
Ponieważ projekt jest jednoosobowy, wszystkie podproblemy realizowane są przez jedną osobę. Tabela poniżej pokazuje kolejność realizacji i wykorzystywane techniki:

Nr	Podproblem	Wykorzystywana technika	Osoba odpowiedzialna	Kolejność
1	Przydział krasnoludków do wyrobisk	Min-Cost Max-Flow (SSP z SPFA)	Daniła Kozik	4.
2	Wyznaczenie trasy patrolu	Otoczka wypukła — Andrew's Monotone Chain	Daniła Kozik	2.
3	Najgłośniejszy krasnoludek na odcinku	Drzewo przedziałowe (Segment Tree) z RMQ	Daniła Kozik	1.
4a	Kompresja tekstu	Kodowanie Huffmana (kolejka priorytetowa)	Daniła Kozik	3.
4b	Wyszukiwanie haseł	Drzewo prefiksowe (Trie)	Daniła Kozik	3.
—	Generator testów, testy jednostkowe	pytest + skrypty pomocnicze	Daniła Kozik	równolegle
—	Integracja modułów, packaging, CI	—	Daniła Kozik	na końcu

Celowo zaczynam od najprostszego modułu (drzewo przedziałowe), żeby ułatwić sobie wejście w projekt, a najtrudniejszy (MCMF) zostawiam na koniec — kiedy szkielet projektu, format danych i testy są już ustabilizowane, a sam jestem „rozpędzony” i znam strukturę kodu.

5. Harmonogram realizacji projektu

Projekt realizowany jest w okresie **5–30 maja 2026** (4 tygodnie). Termin oddania: **30 maja 2026**.
Poniżej tabelaryczny harmonogram przedstawiający rozkład prac:



T1–T4 to kolejne tygodnie maja:

Tydzień	Daty	Główne zadania
T1	5–11 maja	Analiza i specyfikacja, setup repo, Drzewo Przedziałowe, start dokumentacji
T2	12–18 maja	Otoczka wypukła, początek Huffman + Trie, testy modułów 2 i 3
T3	19–25 maja	Dokończenie Huffman + Trie, początek MCMF (najtrudniejszy moduł)
T4	26–30 maja	Dokończenie MCMF, integracja modułów, testy systemowe, finalizacja dokumentacji

Uzasadnienie kolejności modułów

Moduły są szeregowane od najprostszego (drzewo przedziałowe) do najtrudniejszego (MCMF). Pozwala to na stopniowe wchodzenie w projekt i pozostawienie najbardziej wymagającej części na moment, kiedy znam już strukturę projektu i mam działający szkielet testów. Moduł MCMF został oznaczony jako krytyczny — jest najdłuższy (~6 dni) i jednocześnie najbardziej ryzykowny pod kątem opóźnień, dlatego zarezerwowałem mu cały trzeci tydzień plus zapas na początku czwartego.

Założenia czasowe

- ~10–12 godzin tygodniowo (głównie wieczory + jeden większy blok w weekend)
- testy pisane równolegle z każdym modułem (nie zostawiam ich „na koniec”)

- dokumentacja końcowa rozpoczęta już w pierwszym tygodniu (konfiguracja repo, opis specyfikacji), kontynuowana stopniowo i finalizowana w ostatnich dniach
- bufor bezpieczeństwa: integracja kończy się 27 maja, do deadline'u zostają 3 dni na ewentualne poprawki

Ryzyka

- **Moduł MCMF** może wymagać więcej niż 6 dni — ostatecznie zdecydowałem się na prostszą wersję z SPFA (wariant Bellmana-Forda z kolejką), która okazała się wystarczająco szybka na rozmiarach z testów.
- W razie kompletnego utknięcia na jednym module reszta projektu (testy, dokumentacja, integracja pozostałych modułów) i tak posuwa się do przodu — moduły są w dużym stopniu niezależne.

6. Ogólny opis przebiegu prac nad projektem

Sekcja ta opisuje podejście do realizacji projektu, napotkane pułapki oraz faktyczne obserwacje z procesu implementacji.

Organizacja repozytorium

Pracę zaczynam od założenia repozytorium Git na GitHubie. Mimo że jestem jedyną osobą commitującą, świadomie trzymam się prostego modelu z brancha `main` plus oddzielnymi branchami dla każdego modułu (`feature/segment-tree`, `feature/convex-hull`, itd.). Każdy moduł będzie mergowany do `main` dopiero po przejściu testów. To trochę „udawanie zespołu” ze sobą, ale dzięki temu historia repo będzie czytelna i będę mógł się nią posłużyć przy obronie.

Moduł 3 — Drzewo Przedziałowe

Pierwszy do zaimplementowania, bo najprostszy z czterech podproblemów. Zaczynam od niego, żeby zbudować szkielet projektu (struktura katalogów, `pyproject.toml`, podstawowa konfiguracja `pytest`) na łatwiejszym przykładzie. Klasyczna implementacja na liście o rozmiarze $4n$ z rekurencyjnymi `_zbuduj` i `_zapytaj`. Pułapki, na które trzeba uważać:

- off-by-one przy zapytaniach pokrywających pełny zakres $[0, n-1]$ — łatwo schodzi o jeden poziom za daleko w rekursji,
- domyślny limit rekursji w Pythonie wynosi 1000 — przy dużych tablicach trzeba podnieść `sys.setrecursionlimit` albo napisać wersję iteracyjną.

Moduł 2 — Otoczka wypukła

Algorytm Andrew's Monotone Chain — sortowanie leksykograficzne, potem dwukrotne budowanie półotoczki ze sprawdzaniem znaku iloczynu wektorowego. Pułapka klasyczna: punkty współliniowe. Decyduję się je pomijać (nie należą do otoczki w sensie minimalnym), bo do obliczenia obwodu są zbędne — przy okazji upraszcza to porównania znaku iloczynu wektorowego.

Moduł 4 — Huffman + Trie

Realizowany w dwóch krokach. Najpierw Huffman (z `heapq` jako kolejką priorytetową). Tu uważam na **porównywanie krotek w `heapq`**: jeśli w kopcu trzymane są (częstość, węzeł), a dwa węzły mają tę samą częstość, Python próbuje porównać same obiekty `Wezeł` i wywala się z `TypeError`. Rozwiązanie: dodać licznik wstawień jako `tie-breaker` — (częstość, licznik, węzeł). Trie zaimplementowałem jako klasę z `dict` w każdym węźle, a nie tablicą 256-elementową — polskie znaki diakrytyczne (ą, ć, ę itd.) wymuszają Unicode i tablica byłaby marnotrawstwem pamięci.

Moduł 1 — MCMF

Zostawiony na koniec, bo najtrudniejszy. Ostatecznie zdecydowałem się na wersję SSP z SPFA (Shortest Path Faster Algorithm) — wariant Bellmana-Forda z kolejką do znajdowania najtańszej ścieżki powiększającej. SPFA dobrze radzi sobie z ujemnymi kosztami w grafie residualnym (w przeciwieństwie do Dijkstry) i nie wymaga potencjałów Johnsona. Na rozmiarach testowych (do 200 krasnali) działa wystarczająco szybko. Pułapka, na którą trzeba uważać: porównywanie float z `math.inf` przy sprawdzaniu osiągalności ujścia — trzeba inicjalizować tablicę odległości na `inf`, a nie na dużą liczbę.

Integracja

Planuję na końcowy tydzień. Żeby ją uprościć, od początku narzucam sobie wspólny format danych — proste klasy `@dataclass`, brak dziedziczenia między modułami, każdy moduł wystawia jedną funkcję wejściową. Najbardziej oczywisty przypadek brzegowy do obsłużenia: gdy moduł 1 nie przydziela żadnego krasnoludka, moduł 2 dostaje pusty zbiór punktów. Trzeba zadbać, żeby otoczka zwracała wtedy pustą listę i zero, a nie wyrzucała wyjątek.

CI/CD i strategia testowania

Konfiguruję prosty pipeline GitHub Actions — `python:3.11-slim`, `pip install -e .[dev]`, `pytest`. Tylko tyle, ile potrzeba, żeby przy każdym pushu mieć pewność, że zmiany w jednym module nie psują testów w innym. Testy piszę równolegle z każdym modulem, nie zostawiam ich na koniec. Generator losowych danych (`testy/generator.py`) tworzę już w pierwszym tygodniu — bez zespołu, w którym ktoś inny patrzy świeżym okiem na kod, generator losowy jest jedynym sposobem na znalezienie błędów, których sam nie przewidziałem.

Faktyczne obserwacje z realizacji

Największym wyzwaniem okazała się nie implementacja pojedynczych algorytmów, lecz integracja modułów w spójny pipeline — uzgodnienie formatów danych między modułami, obsługa przypadków brzegowych (np. pusty zbiór użytkowanych kopalni) i zaokrąglenia zmiennoprzecinkowe przy współrzędnych. Zmiana RMQ z zwracania samej wartości na zwracanie pary (wartość, indeks) wymusiła modyfikację zarówno drzewa przedziałowego, jak i wszystkich testów — dobra lekcja o wpływie specyfikacji na kod. Dla MCMF zdecydowałem się ostatecznie na SPFA zamiast planowanych potencjałów Johnsona — prostsza implementacja, a testy stresowe (do 200 krasnali) potwierdziły wystarczającą wydajność. Generator stresowy (`testy/generator.py`) pomógł wykryć dwa edge case'y, które nie wyszły w testach jednostkowych.

7. Zastosowane algorytmy i ich złożoność obliczeniowa

7.1. Moduł 1 — MCMF (Min-Cost Max-Flow)

Problem: maksymalne dopasowanie w grafie dwudzielnym z pojemnościami i kosztami (odległościami).

Konstrukcja sieci przepływowej:

- źródło s , ujście t ,
- krawędź $s \rightarrow k_i$ o przepustowości 1 i koszcie 0 dla każdego krasnoludka,
- krawędź $k_i \rightarrow w_j$ o przepustowości 1 i koszcie $D[i][j]$, jeśli krasnoludek k_i potrafi wydobywać minerał z wyrobiska w_j ,
- krawędź $w_j \rightarrow t$ o przepustowości c_j (pojemność wyrobiska) i koszcie 0.

Algorytm: Successive Shortest Paths z SPFA (Shortest Path Faster Algorithm) do znajdowania najtańszej ścieżki powiększającej. SPFA to wariant Bellmana-Forda z kolejką — naturalnie obsługuje ujemne koszty w grafie residualnym.

Złożoność czasowa: $O(F \cdot V \cdot E)$, gdzie $F \leq n$ (SPFA w najgorszym przypadku $O(V \cdot E)$ na iterację).

Złożoność pamięciowa: $O(V + E)$.

7.2. Moduł 2 — Otoczka wypukła (Andrew's Monotone Chain)

Problem: dla zbioru n punktów na płaszczyźnie znaleźć $\text{conv}(P)$.

Algorytm: sortowanie leksykograficzne, następnie dwukrotne zbudowanie półotoczki (dolnej i górnej) ze sprawdzaniem znaku iloczynu wektorowego.

Złożoność czasowa: $O(n \log n)$ — zdominowana przez sortowanie.

Złożoność pamięciowa: $O(n)$.

Rozwazałem też algorytm Grahama, ale Monotone Chain jest prostszy w implementacji i ma mniej przypadków brzegowych — szczególnie istotne, gdy nie ma kogo zapytać, czy moja obsługa edge case'ów jest poprawna.

7.3. Moduł 3 — Drzewo przedziałowe (RMQ)

Problem: statyczne zapytania o maksimum na przedziale.

Algorytm: drzewo przedziałowe oparte na liście o rozmiarze $4n$.

Budowa: $O(n)$ czasowo, $O(n)$ pamięciowo.

Pojedyncze zapytanie: $O(\log n)$.

Rozwazałem też **Sparse Table** ($O(n \log n)$ budowa, $O(1)$ zapytanie), które byłoby teoretycznie lepsze przy dużej liczbie zapytań, ale drzewo przedziałowe daje możliwość rozszerzenia o aktualizacje (gdyby kiedyś pojawiła się dynamiczna głośność), więc zostałem przy nim. To była świadoma decyzja.

7.4. Moduł 4 — Huffman + Trie

Kodowanie Huffmana

- Algorytm: zachłanne łączenie dwóch najmniej częstych symboli, z kolejką priorytetową (heapq).
- Czasowa: $O(\sigma \log \sigma)$, gdzie σ to liczba unikalnych symboli.
- Pamięciowa: $O(\sigma)$.
- Sama kompresja/dekompresja tekstu o długości N : $O(N)$ przy znanym kodzie.

Trie (drzewo prefiksowe)

- Wstawienie słowa o długości L : $O(L)$.
- Wyszukanie słowa: $O(L)$.
- Pamięciowa: $O(\text{TotalLen})$ przy implementacji z dict w każdym węźle.

8. Poprawność rozwiązania

Testowanie podzieliłem na trzy poziomy:

Testy jednostkowe (pytest)

Każdy moduł ma własny zestaw testów weryfikujących pojedyncze funkcje. Napisano 26 testów jednostkowych (5 MCMF, 3 otoczka, 9 RMQ, 9 Huffman+Trie). Używam `pytest.parametrize` do tablicowania wielu wariantów wejścia.

Testy integracyjne

Sprawdzają, czy moduły poprawnie przekazują sobie dane. Najważniejszy z nich uruchamia cały pipeline na małym, ręcznie zweryfikowanym wejściu (8 krasnoludków, 3 wyrobiska, 12 dekametrowców, krótki tekst).

Testy stresowe

Generator losowych wejść (oddzielny moduł `testy/generator.py`) tworzy duże instancje (do 1000 krasnoludków). Wyniki są porównywane albo z naiwną implementacją (np. brute-force MCMF dla małych n), albo z niezmiennikami (np. obwód otoczki nie maleje przy dodaniu punktu na zewnątrz).

Sprawdzane przypadki brzegowe

- **Otoczka wypukła:** zbiór pusty, jeden punkt, dwa punkty (degeneracja do odcinka), trzy punkty współliniowe (zwracam odcinek od skrajnie lewego do skrajnie prawego), wszystkie punkty pokrywające się, duża liczba punktów współliniowych z jednym wystającym.
- **Drzewo przedziałowe:** zapytanie o jeden element ($l = r$), zapytanie o cały zakres ($l = 0, r = n - 1$), zapytanie z $l > r$ (zwracam wartość neutralną `float('-inf')`), tablica długości 1, tablica długości potęgi dwójki vs nie-potęgi.
- **MCMF:** brak krasnoludków znających dany minerał, więcej krasnoludków niż łącznej pojemności wyrobisk, wszystkie odległości równe, graf rozłączony.
- **Huffman:** pusty tekst, tekst z jednym unikalnym znakiem (przypadek graniczny — drzewo Huffmana z jednym liściem; obsługę go specjalnie, przypisując kod „0”), tekst gdzie wszystkie znaki mają tę samą częstość, tekst z polskimi znakami (UTF-8 obsługiwane natywnie).
- **Trie:** wstawianie pustego słowa, wstawianie słowa będącego prefiksem innego, wyszukiwanie słowa, którego nie ma w drzewie, polskie znaki diakrytyczne.

Argumenty poprawności (skrócone)

- **MCMF:** poprawność wynika z twierdzenia o maksymalnym przepływie minimalnego kosztu — algorytm SSP zwraca przepływ optymalny, jeśli w grafie residualnym nie istnieje cykl o ujemnym koszcie, co jest gwarantowane przez wybieranie ścieżek o najmniejszym koszcie w każdej iteracji.
- **Otoczka wypukła:** klasyczny dowód indukcyjny — niezmiennikiem pętli jest, że stos zawiera punkty tworzące poprawny lewoskrętny łańcuch.
- **Drzewo przedziałowe:** indukcja po wysokości drzewa, niezmiennik węzła: przechowuje maksimum na swoim podprzedziale.
- **Huffman:** klasyczny dowód optymalności kodu prefiksowego (twierdzenie Huffmana — kod jest optymalny w klasie kodów prefiksowych).
- **Trie:** struktura z definicji — każde słowo wstawione zostaje znalezione (trywialnie po konstrukcji).

9. Rodzaj wykorzystanej technologii, języków programowania

- **Język główny: Python 3.11+.** Wybrałem go z kilku powodów: czytelność kodu (ważne, gdy pracuję sam), bogata biblioteka standardowa (`heapq`, `collections.defaultdict`, `dataclasses`), oraz znajomość tego języka z poprzednich kursów. Świadomie zrezygnowałem z C++, mimo

że dawałby kilkukrotnie lepszą wydajność — uznałem, że nawet w najgorszym module (MCMF) Python jest „wystarczająco szybki” na rozmiary danych podane w specyfikacji.

- **Zarządzanie zależnościami:** `pyproject.toml` (PEP 621) + `pip` w środowisku wirtualnym (`venv`).
- **Testowanie:** `pytest` — testy jednostkowe i integracyjne, z `pytest.parametrize` i `pytest.fixture`.
- **Profilowanie:** `cProfile` (zlokalizowanie wąskich gardeł w MCMF) i `tracemalloc` (sprawdzanie referencji w drzewie Huffmana).
- **Formatowanie i lintowanie:** `black` + `ruff`.
- **Adnotacje typów:** w pełni używam `typing`, uruchamiam `mypy --strict` jako część pipeline'u.
- **Kontrola wersji:** `Git` + `GitHub`. Branche `feature/<nazwa-modułu>`, merge do `main` po przejściu testów.
- **CI/CD:** `GitHub Actions` z prostym `.github/workflows/ci.yml` — instaluje zależności na obrazie `python:3.11-slim` i uruchamia `pytest` + `mypy` + `ruff`.

Konwencja nazewnicza: zdecydowałem się na polskie nazwy klas i metod tam, gdzie nie kłóci się to z biblioteką standardową. Przykład:

```
from dataclasses import dataclass
from typing import Optional
```

```
@dataclass
class Krawedz:
    do_: int
    przepustowosc: int
    koszt: int
    odwrotna: int # indeks krawędzi odwrotnej
```

```
@dataclass
class Wierzcholek:
    x: float
    y: float
    id_wyrobiska: int
```

```
class DrzewoPrzedzialowe:
    def __init__(self, glosnosci: list[int]) -> None:
        self.rozmiar = len(glosnosci)
        self.wezly: list[float] = [float('-inf')] * (4 * self.rozmiar)
        self._zbuduj(1, 0, self.rozmiar - 1, glosnosci)

    def zapytaj_maks(self, lewy: int, prawy: int) -> float:
        if lewy > prawy:
            return float('-inf')
        return self._zapytaj(1, 0, self.rozmiar - 1, lewy, prawy)
```

Uznałem, że spójność polskich nazw z polskojęzyczną dokumentacją ułatwia mi nawigację w kodzie podczas obrony.

10. Informacja dodatkowa

Patrząc z perspektywy końca projektu — najwięcej uczę się nie z samego zaimplementowania pojedynczych algorytmów (MCMF, otoczka, drzewo przedziałowe, Huffman, Trie znam wszystkie z

wykładu i ćwiczeń), tylko ze sklejenia ich w jeden działający system. Kiedy moduł 2 zaczyna brać dane z modułu 1, nagle okazuje się, że trzeba uzgodnić format, obsłużyć przypadki puste, zapanować nad zaokrągleniami liczb zmiennoprzecinkowych przy współrzędnych wyrobisk, pomyśleć o testach end-to-end zamiast tylko jednostkowych.

Drugi wniosek, specyficzny dla pracy w pojedynkę: warto wcześniej pisać generator losowych testów i pisać testy zaraz po napisaniu funkcji, a nie „kiedyś potem”. Bez kolegi z zespołu, który spojrzy świeżym okiem, generator losowych danych jest jedynym sposobem na znalezienie błędów, których sam nie przewidziałem.

Trzeci wniosek dotyczy wyboru języka. Świadomy wybór Pythona zamiast C++ na początku wydawał mi się lekko ryzykowny — wiedziałem, że MCMF będzie wolniejszy. Okazało się jednak, że oszczędność czasu programisty (mniej boilerplate'u, brak segfaultów, łatwiejsze debugowanie) przeważa nad różnicą wydajności. Dla jednoosobowej pracy Python był zdecydowanie lepszym wyborem.

Główny element, na który warto zwrócić uwagę przy realizacji: wcześniejsze zaplanowanie integracji modułów. Chcę uniknąć sytuacji, w której każdy moduł powstaje w izolacji, a integracja zostaje na sam koniec — dlatego od początku trzymam się wspólnych typów danych narzuconych przez `@dataclass` i jasno zdefiniowanych funkcji wejściowych dla każdego modułu.

Toruń, maj 2026 — Danila Kozik